

PYTHON

Module 6: Documentation & Comments

CODEFIVE

The Purpose of Comments

In professional software development at CodeHive, code is read much more often than it is written. Comments are non-executable lines that serve three vital roles:

1. **Clarification:** Explaining complex logic or 'why' a specific approach was taken.
2. **Readability:** Breaking up code into logical sections for human eyes.
3. **Debugging:** Temporarily disabling code blocks during testing without deleting them.

Single-Line Comments

The most common way to comment in Python is using the hash character (`#`). Everything following the hash on that line is ignored by the Python interpreter.

Placement Options:

- **Stand-alone:** `# Initialize the counter` (placed above the code).
- **In-line:** `x = 10 # This is an end-of-line comment` .

In-line comments should be used sparingly to avoid cluttering the horizontal space of your script.

Managing Multiline Comments

Strictly speaking, Python does not have a dedicated syntax for multiline comments (like `/* */` in C++). Developers have two main workarounds:

The Block Method: Adding an individual `#` to the start of every line. This is the official PEP 8 recommendation.

Example:

```
# Line one
```

```
# Line two
```

This ensures that the interpreter clearly skips every intended line.

The Docstring Workaround

You can also use Triple Quotes (`"""` or `'''`) to create multi-line comments. Technically, these are multiline strings.

If a string literal is not assigned to a variable, Python processes it but then discards it. This effectively acts as a comment block.

Note of Caution: While useful, docstrings (triple quotes) are usually reserved for function and class documentation. For general notes, the `#` block method is preferred.

Preventing Code Execution

Comments are an essential tool for 'commenting out' code. If you have a bug, you can disable specific lines to isolate the issue.

Scenario:

```
# print(database.delete_all())
```

```
print('Database operation skipped for safety')
```

This allows developers to keep code fragments for later use without them affecting the current program run.

Best Practices (PEP 8)

CodeHive follows the Python Enhancement Proposal (PEP 8) standards for clean code:

- Use full sentences starting with a capital letter (unless the first word is an identifier).
- Ensure there is a space after the # before the text begins.
- Keep comments up to date. An outdated comment is worse than no comment at all.
- Avoid 'Obvious Comments' like
`x = 1 # assign 1 to x .`

Summary & Implementation

Comments are the documentation that lives inside your script. They turn a raw script into a professional piece of software.

Key Takeaway:

Code tells the computer 'How'.

Comments tell the human 'Why'.

Mastering this balance is the hallmark of a great CodeHive developer.